CSE 344 – Homework #5

Priority-Based Cargo Delivery Simulation

Name: Ayşe Feyza SERBEST
Student ID: 220104004052
Course: CSE 344 – System Programming
Date: 10 May 2026

Contents

1 System Design

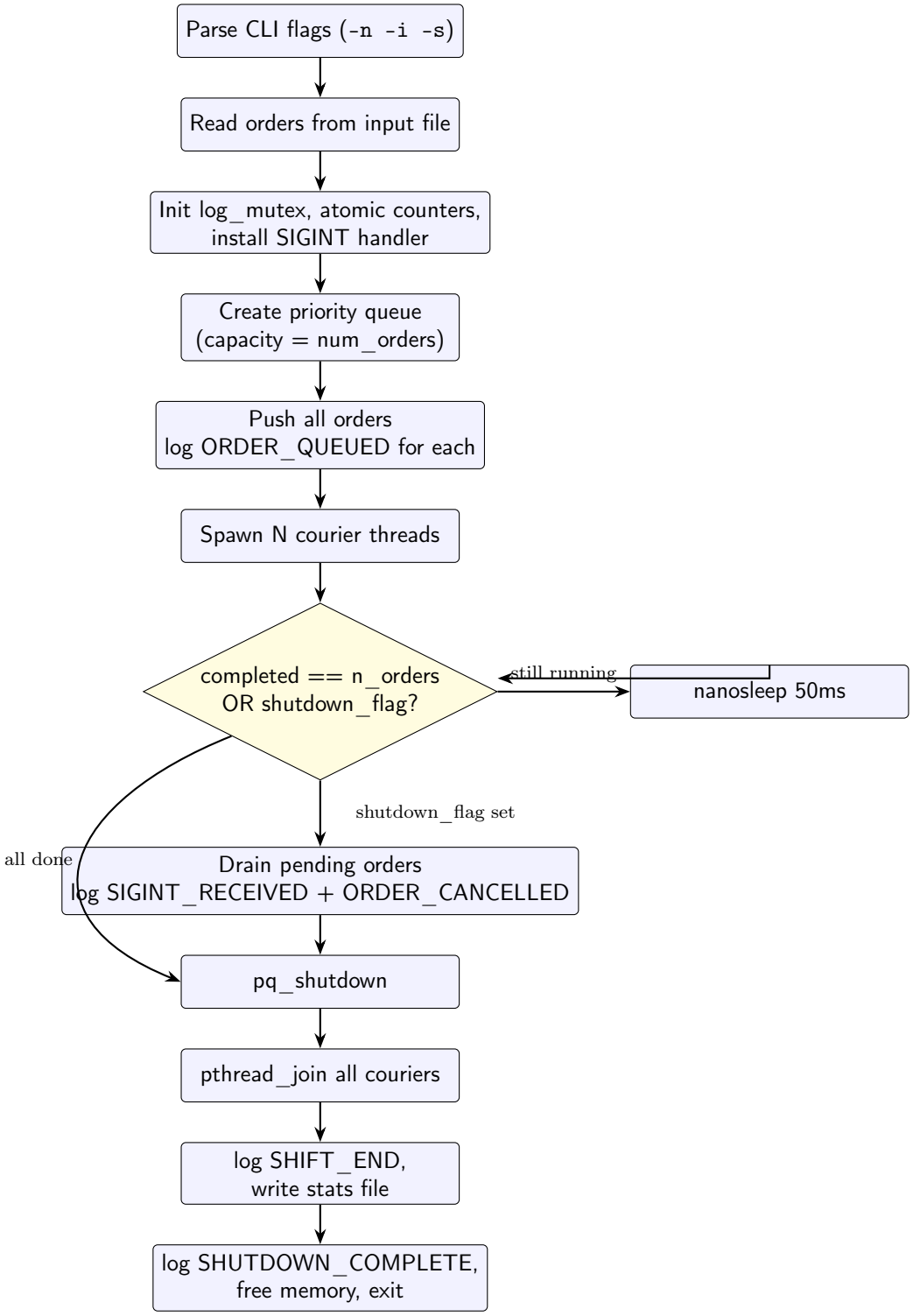
1.1 Module structure

The program is decomposed into eight small modules so each piece can be reasoned about independently. The split follows the classic single-responsibility principle: parsing, queueing, scheduling, signal handling, statistics, and logging are completely separate concerns.

Module	Files	Responsibility
Order definitions	`order.h/c`	`Order` struct, `Priority` enum, string conversions
Parser	`parser.h/c`	Read input file, produce a clean `Order` array
Priority queue	`priority_queue.h/c`	Thread-safe min-heap with mutex + cond var
Courier	`courier.h/c`	Per-thread main loop (pop → deliver → repeat)
Statistics	`stats.h/c`	C11 atomic counters, per-courier stats, report writer
Logging	`log.h/c`	Thread-safe `printf` wrapper guarded by `log_mutex`
Signal handling	`signal_handler.h/c`	Async-signal-safe SIGINT handler that only flips a flag
Main	`main.c`	CLI parsing, lifecycle orchestration

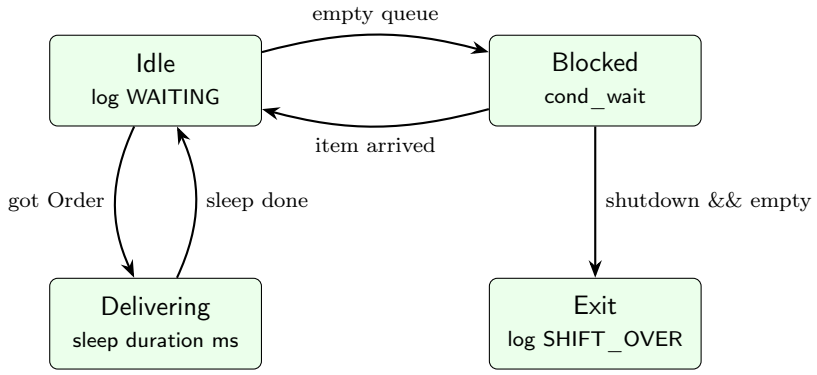
1.2 Lifecycle overview

The system has three phases: **setup**, **shift**, and **shutdown**. Threads are created exactly once at the start of *shift* and destroyed exactly once at the end of *shutdown*. No thread is ever spawned per-order, which is what the homework requires (and explicitly penalises if violated).



1.3 Courier thread lifecycle

Each courier runs the same simple loop. The blocking pop hides all complexity: when the queue is empty, the courier sleeps on a condition variable and consumes zero CPU. When `pq_shutdown` is signalled, the pop returns `PQ_POP_SHUTDOWN` and the loop exits.



The crucial property is that no courier thread is ever recreated. The pool size is fixed at `-n` and remains constant for the entire program. When orders run out, threads exit through the SHUTDOWN path; they are not waiting to be respawned.

1.4 Why this shape?

A few design alternatives were rejected:

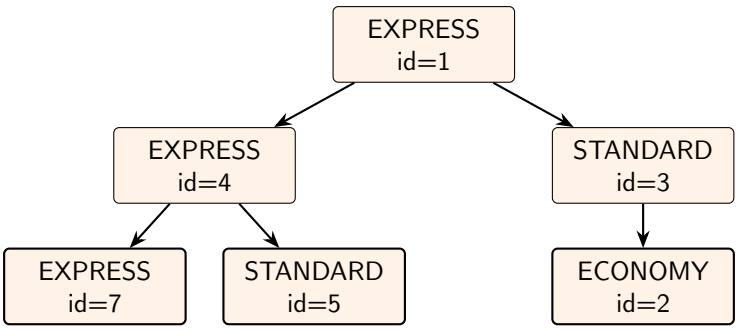
- **One thread per order:** trivial to write but defeats the point of a thread pool, and is explicitly penalised (−80).
- **Single-threaded simulation iterating over orders:** violates the concurrency requirement (−80).
- **Producer/consumer with main feeding workers:** unnecessary indirection. Orders are all known up-front, so they are loaded into the queue once and only consumed afterward. This collapses to a consumer-only model which is simpler to lock and to reason about.

2 Priority Queue Design

2.1 Data structure

The queue is a **binary min-heap** stored in a flat array. Heaps give logarithmic push and pop, are cache-friendly because the array is contiguous, and have no per-item allocation overhead.

The heap is **0-indexed**: for a slot at index i, the parent is at $(i-1)/2$, the left child at $2i+1$, and the right child at $2i+2$. This trades the slightly cleaner 1-indexed math for not wasting heap slot zero.



When `pq_pop_blocking` extracts the root, the last element fills slot 0 and is sifted down until the heap property is restored. Every push appends at the end and sifts up. Both operations are $O(\log n)$.

2.2 The comparison function – where priority lives

The heap property is defined entirely by a single `static bool order_less(a, b)` function. Everything else (push, pop, drain, sift_up, sift_down) is generic. The ordering rule from the homework is encoded *only* here:

```
static bool order_less(const Order *a, const Order *b) {
    if (a->priority != b->priority) return a->priority < b->priority;
    return a->id < b->id; // tie-breaker: smaller id first
}
```

`Priority` is defined as an enum with `EXPRESS=1`, `STANDARD=2`, `ECONOMY=3`. Smaller numeric value means higher priority, so the comparison reads naturally: a smaller priority enum or, when tied, a smaller id wins. This satisfies both "EXPRESS > STANDARD > ECONOMY" and the homework's tie-breaker ("the one with the lower id is processed first").

If the homework's priority rules ever changed, only this single function would need to change. The rest of the queue is completely insulated from policy.

2.3 Fixed capacity, value semantics

The queue stores `Order` *by value*, not by pointer. Each `Order` is small (about 48 bytes) and fixed-size, so copying is cheap; in return, the queue owns no external memory and the caller cannot dangle a pointer by freeing an order while a courier still has a reference to it.

The capacity is fixed at construction time and equals the number of parsed orders. Since the homework guarantees that all orders are loaded before any courier runs, no growth is needed and overflow is impossible by construction. The `pq_push` overflow check exists only as defence in depth.

2.4 Drain operation for cancellation

In addition to push and pop, the queue exposes `pq_drain`, which extracts every remaining element *in priority order*. This is used during SIGINT handling: pending orders need to be reported as cancelled in the same priority order they would have been delivered, not the arbitrary order in which the heap stores them internally. The implementation calls extract-min in a loop, which gives sorted output in $O(n \log n)$.

3 Synchronization Strategy

The program has **four** synchronization primitives, each protecting exactly one resource. Each is small and focused, so it is easy to verify by inspection that the locking discipline is consistent.

Primitive	Type	Protects
`pq->mutex`	`pthread_mutex_t`	heap[], size, shutdown flag (inside the queue)
`pq->not_empty`	`pthread_cond_t`	signals state change for the mutex-protected predicate
`log_mutex`	`pthread_mutex_t`	stdout (one whole line at a time)
`g_shutdown_requested`	`volatile sig_atomic_t`	cross-context flag (signal handler ↔ main)

3.1 The queue mutex and condition variable

The queue protects a small piece of state – the heap array, its size, and a shutdown boolean – with a single mutex. There is no fine-grained locking; the whole queue is locked for the entire duration of every operation. This is the right call for a structure that is touched only on push/pop boundaries (typically a few microseconds of work each).

The condition variable `not_empty` keeps couriers from busy-waiting. The pop function follows the canonical pattern:

```
lock(mutex);
while (size == 0 && !shutdown)
    cond_wait(not_empty, mutex); // releases mutex, sleeps, re-acquires
if (size == 0) { // must be shutdown
    unlock(mutex); return SHUTDOWN;
}
extract root, sift down;
unlock(mutex); return OK;
```

The `while` loop (rather than an `if`) handles spurious wakeups, which `pthread_cond_wait` is allowed to produce per POSIX. Without the loop, a spuriously woken courier could read a still-empty queue.

`pq_push` calls `cond_signal(not_empty)` to wake exactly one waiting courier. `pq_shutdown` calls `cond_broadcast(not_empty)` to wake them all, because every blocked courier needs to re-check and exit.

3.2 The log mutex

Multiple courier threads write to stdout concurrently, and `printf` itself is not safe to interleave with another `printf` from a different thread within a single line. The homework explicitly grades on this and gives −20 for interleaved output.

The solution is a dedicated `log_mutex` and a single `log_printf(fmt, ...)` helper that:

1. Acquires the mutex,
2. `vprintf`s the formatted line,
3. Writes a newline,
4. Calls `fflush(stdout)` so the line hits the OS atomically,
5. Releases the mutex.

Every output line in the program goes through this helper. The `fflush` is important: without it, line-buffered stdout could split a line across two `write()` syscalls when redirected, which on a pipe could still appear interleaved. Flushing inside the lock guarantees the whole line is one syscall.

3.3 The SIGINT flag

The shutdown flag uses neither a mutex nor C11 atomics; it is a `volatile sig_atomic_t`. The two attributes serve different purposes:

- `sig_atomic_t` is the POSIX-guaranteed type for an integer that can be safely loaded and stored from a signal handler. Other types (e.g. `int`) work in practice on common platforms but are not portable.
- `volatile` prevents the compiler from caching the variable in a register inside main's polling loop. Without `volatile`, an aggressive optimiser could hoist the read out of the loop and main would never observe the flip.

Using a mutex inside a signal handler would be unsafe (mutexes are not async-signal-safe). Using `_Atomic int` would also work, but `sig_atomic_t` is the canonical choice and makes the intent obvious.

3.4 Atomic counters

The three statistics counters – `completed_orders`, `cancelled_orders`, `total_delivery_time` – are declared `_Atomic` and updated with `atomic_fetch_add`. The homework requires this and penalises −20 for using a mutex instead.

The choice is correct because each update is a single read-modify-write of a single integer, which is exactly what `atomic_fetch_add` provides. There is no compound invariant across multiple counters that would require them to be updated together; each event independently increments one (or two) counters.

Per-courier statistics (`CourierStats`) are *not* atomic, and intentionally so. Each `CourierStats` is owned by exactly one courier thread for the entire shift, so there is no concurrent access. Main reads them only after `pthread_join`, which provides a synchronizes-with relation per POSIX, so the read sees the final values without any explicit synchronization.

4 SIGINT Handling

The homework gives two acceptable SIGINT-handling patterns: `sigwait()` in a dedicated thread, or a signal handler that only sets a flag. This implementation uses the **signal-handler-with-flag** pattern.

4.1 The handler itself

The handler is reduced to a single line:

```
static void sigint_handler(int signum) {
    (void)signum;
    g_shutdown_requested = 1;
}
```

Anything more would risk calling an async-signal-unsafe function. `printf`, `pthread_*`, `malloc`, `free`, locking, even `exit` are not on the POSIX async-signal-safe list. The penalty for using an unsafe operation here is −20. By writing exactly one variable of the right type, this handler is provably safe.

`sigaction` is used (not `signal`) because `sigaction` has well-defined semantics across platforms; `signal` does not. `SA_RESTART` is deliberately *not* set: without it, a `nanosleep` interrupted by SIGINT returns `EINTR` rather than continuing to sleep. This lets main observe the flag within microseconds rather than waiting up to 50ms for the next poll, and it lets the courier's delivery sleep loop resume the *remaining* time after `EINTR`.

4.2 Step-by-step shutdown sequence

1. **User presses Ctrl-C.** The kernel delivers SIGINT to the process; some thread runs the handler, which sets `g_shutdown_requested = 1` and returns. Any `nanosleep` currently blocked returns `EINTR`.
2. **Main observes the flag.** Its polling loop wakes from `nanosleep`, the next iteration evaluates the predicate as false, and the loop exits.
3. **Main drains the queue.** `pq_drain` removes every remaining order in priority order. For each, main logs `ORDER_CANCELLED` and bumps the `cancelled_orders` atomic counter.
4. **Main signals shutdown.** `pq_shutdown` flips the queue's shutdown flag and broadcasts on the condition variable, so any couriers still blocked in `pq_pop_blocking` wake up.
5. **Active deliveries finish.** Couriers in the middle of a `nanosleep` during a delivery hit `EINTR`; the EINTR-safe wrapper resumes the leftover time, so the full delivery duration still elapses.
6. **Couriers exit.** On their next `pq_pop_blocking` call, they observe an empty queue with shutdown set and return `PQ_POP_SHUTDOWN`. They log `SHIFT_OVER` and return.
7. **Main joins all couriers.** Then it logs `SHIFT_END`, writes the stats file, logs `SHUTDOWN_COMPLETE`, frees memory, and returns `EXIT_SUCCESS`.

4.3 Why deliveries cannot be interrupted

The courier sleeps for `duration` × `500ms` per delivery using `nanosleep`. When SIGINT arrives, `nanosleep` returns −1 with `errno == EINTR` and writes the remaining time to its second argument. The wrapper handles this:

```
while (nanosleep(&req, &rem) == -1) {
    if (errno != EINTR) break;
    req = rem; // sleep the leftover, again
}
```

A courier that was 100ms into a 4000ms delivery when SIGINT fires will resume sleeping for 3900ms. The total wall-clock spent in the delivery is unchanged, the `DELIVERY_COMPLETE` line still logs the full duration, and the atomic counter still increments by the full amount. The homework's "a delivery cannot be interrupted once started" rule is satisfied at the syscall level.

4.4 Race conditions considered

The interesting race is between `pq_size` (reporting N pending) and `pq_drain` (returning M pending). Couriers can pop between those calls, so $M \leq N$; but no thread ever pushes after the initial setup, so M can never exceed what was pending when the snapshot was taken. The implementation logs `pending_orders` using M (the actual drained count) rather than N, which is what the `ORDER_CANCELLED` lines need to match anyway. There is no inconsistency window visible to the user.

5 Test Scenarios

All five mandatory tests from PDF section 7 were run on Debian 12 (64-bit). Uncut console outputs follow each scenario.

5.1 Test 1 – `-n 3 -i 10orders_mix.txt`

What it shows: all ten orders are picked up exactly once, EXPRESS-then-STANDARD-then-ECONOMY ordering is preserved, ties within a priority class are broken by ascending id, all couriers reach `SHIFT_OVER`, and the stats file at the end matches the global counters.

Test 1 – console output

```
<$  Hw5 git:(main)  ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=5
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Acar priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Burak_Turan priority=EXPRESS duration=4
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Koc priority=STANDARD duration=13
[CARGOGTU] ORDER_QUEUED id=9 recipient=Can_Ozkan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Arslan priority=EXPRESS duration=6
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=7 recipient=Burak_Turan priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=7 recipient=Burak_Turan duration=2000ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=10 recipient=Merve_Arslan priority=EXPRESS
[COURIER-3] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=2500ms
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=10 recipient=Merve_Arslan duration=3000ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=8 recipient=Elif_Koc priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=6000ms
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=6 recipient=Zeynep_Acar priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=8 recipient=Elif_Koc duration=6500ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=9 recipient=Can_Ozkan priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=6 recipient=Zeynep_Acar duration=7500ms
[COURIER-1] WAITING
[COURIER-2] DELIVERY_COMPLETE id=9 recipient=Can_Ozkan duration=5500ms
[COURIER-2] WAITING
[COURIER-3] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=10000ms
[COURIER-3] WAITING
[COURIER-1] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=51500ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

5.2 Test 2 – `-n 10 -i 10orders_economy.txt`

What it shows: with all ten orders at the same `ECONOMY` priority, the tie-breaker rule must produce ascending-id order in `DELIVERY_START` lines; multiple couriers participate (since there are enough threads available); no id appears twice. This scenario specifically tests the tie-break path of `order_less`, which the more diverse Test 1 does not exercise cleanly.

Test 2 – console output

```
  Hw5 git:(main)  ./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=12
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=ECONOMY duration=5
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=ECONOMY duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Acar priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Burak_Turan priority=ECONOMY duration=4
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Koc priority=ECONOMY duration=13
[CARGOGTU] ORDER_QUEUED id=9 recipient=Can_Ozkan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Arslan priority=ECONOMY duration=6
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=ECONOMY
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-4] WAITING
[COURIER-4] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-5] WAITING
[COURIER-5] DELIVERY_START id=4 recipient=Fatma_Sahin priority=ECONOMY
[COURIER-7] WAITING
[COURIER-7] DELIVERY_START id=5 recipient=Ali_Celik priority=ECONOMY
[COURIER-8] WAITING
[COURIER-6] WAITING
[COURIER-9] WAITING
[COURIER-9] DELIVERY_START id=7 recipient=Burak_Turan priority=ECONOMY
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=9 recipient=Can_Ozkan priority=ECONOMY
[COURIER-8] DELIVERY_START id=6 recipient=Zeynep_Acar priority=ECONOMY
[COURIER-10] WAITING
[COURIER-10] DELIVERY_START id=10 recipient=Merve_Arslan priority=ECONOMY
[COURIER-6] DELIVERY_START id=8 recipient=Elif_Koc priority=ECONOMY
[COURIER-9] DELIVERY_COMPLETE id=7 recipient=Burak_Turan duration=2000ms
[COURIER-9] WAITING
[COURIER-5] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=2500ms
[COURIER-5] WAITING
[COURIER-10] DELIVERY_COMPLETE id=10 recipient=Merve_Arslan duration=3000ms
[COURIER-10] WAITING
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] WAITING
[COURIER-7] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-7] WAITING
[COURIER-2] DELIVERY_COMPLETE id=9 recipient=Can_Ozkan duration=5500ms
[COURIER-2] WAITING
[COURIER-4] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=6000ms
[COURIER-4] WAITING
[COURIER-6] DELIVERY_COMPLETE id=8 recipient=Elif_Koc duration=6500ms
[COURIER-6] WAITING
[COURIER-8] DELIVERY_COMPLETE id=6 recipient=Zeynep_Acar duration=7500ms
[COURIER-8] WAITING
[COURIER-3] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=10000ms
[COURIER-3] WAITING
[COURIER-9] SHIFT_OVER
[COURIER-5] SHIFT_OVER
[COURIER-10] SHIFT_OVER
```

```
[COURIER-1] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=51500ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

5.3 Test 3 – `-n 2 -i 20orders_mix.txt`, send SIGINT after ∼2 seconds

What it shows: after Ctrl-C is pressed, the program logs `SIGINT_RECEIVED pending_orders=N` followed by exactly N `ORDER_CANCELLED` lines in priority order. Any couriers in the middle of a delivery finish that delivery before exiting, so the `DELIVERY_COMPLETE` lines for in-flight orders appear after the `ORDER_CANCELLED` block. `SHIFT_END completed=X cancelled=Y` satisfies $X + Y = 20$.

Test 3 – console output

```
  Hw5 git:(main)  ./cargoGTU -n 2 -i 20orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=2 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=5
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Acar priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Burak_Turan priority=EXPRESS duration=4
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Koc priority=STANDARD duration=13
[CARGOGTU] ORDER_QUEUED id=9 recipient=Can_Ozkan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Arslan priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=11 recipient=Hasan_Kurt priority=STANDARD duration=7
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Yildiz priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=13 recipient=Emre_Dogan priority=EXPRESS duration=10
[CARGOGTU] ORDER_QUEUED id=14 recipient=Derya_Cetin priority=STANDARD duration=8
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kerem_Gunes priority=ECONOMY duration=14
[CARGOGTU] ORDER_QUEUED id=16 recipient=Melis_Aydin priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=17 recipient=Yusuf_Sari priority=STANDARD duration=16
[CARGOGTU] ORDER_QUEUED id=18 recipient=Ece_Kaplan priority=ECONOMY duration=22
[CARGOGTU] ORDER_QUEUED id=19 recipient=Omer_Polat priority=EXPRESS duration=9
[CARGOGTU] ORDER_QUEUED id=20 recipient=Deniz_Yaman priority=STANDARD duration=5
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=2500ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=7 recipient=Burak_Turan priority=EXPRESS
^C[CARGOGTU] SIGINT_RECEIVED pending_orders=17
[CARGOGTU] ORDER_CANCELLED id=10 recipient=Merve_Arslan priority=EXPRESS
[CARGOGTU] ORDER_CANCELLED id=13 recipient=Emre_Dogan priority=EXPRESS
[CARGOGTU] ORDER_CANCELLED id=16 recipient=Melis_Aydin priority=EXPRESS
[CARGOGTU] ORDER_CANCELLED id=19 recipient=Omer_Polat priority=EXPRESS
[CARGOGTU] ORDER_CANCELLED id=3 recipient=Mehmet_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=5 recipient=Ali_Celik priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=8 recipient=Elif_Koc priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=11 recipient=Hasan_Kurt priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=14 recipient=Derya_Cetin priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=17 recipient=Yusuf_Sari priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=20 recipient=Deniz_Yaman priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=2 recipient=Ayse_Kaya priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=6 recipient=Zeynep_Acar priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=9 recipient=Can_Ozkan priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=12 recipient=Selin_Yildiz priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=15 recipient=Kerem_Gunes priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=18 recipient=Ece_Kaplan priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-2] WAITING
[COURIER-2] SHIFT_OVER
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Burak_Turan duration=2000ms
[COURIER-1] WAITING
[COURIER-1] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=3 cancelled=17 total_time=8500ms
```

```
[CARGOGTU] SHUTDOWN_COMPLETE
```

5.4 Test 4 – `-n 3 -i 10orders_mix.txt` under Valgrind

What it shows: `valgrind -leak-check=full ./cargoGTU` finishes with 0 errors and 0 leaks (specifically `definitely lost: 0 bytes`, `indirectly lost: 0 bytes`, `possibly lost: 0 bytes`, and `ERROR SUMMARY: 0 errors from 0 contexts`). The clean Valgrind result follows from a few deliberate choices: every `malloc`/`calloc` has a matching `free`, every mutex/cond is destroyed, every thread is joined.

Test 4 – console output

```
  Hw5 git:(main)  valgrind ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
==3181== Memcheck, a memory error detector
==3181== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==3181== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==3181== Command: ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
==3181==
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=5
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Acar priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Burak_Turan priority=EXPRESS duration=4
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Koc priority=STANDARD duration=13
[CARGOGTU] ORDER_QUEUED id=9 recipient=Can_Ozkan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Arslan priority=EXPRESS duration=6
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=7 recipient=Burak_Turan priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=7 recipient=Burak_Turan duration=2000ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=10 recipient=Merve_Arslan priority=EXPRESS
[COURIER-3] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=2500ms
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=10 recipient=Merve_Arslan duration=3000ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=8 recipient=Elif_Koc priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=6000ms
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=6 recipient=Zeynep_Acar priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=8 recipient=Elif_Koc duration=6500ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=9 recipient=Can_Ozkan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=6 recipient=Zeynep_Acar duration=7500ms
[COURIER-3] WAITING
[COURIER-2] DELIVERY_COMPLETE id=9 recipient=Can_Ozkan duration=5500ms
[COURIER-2] WAITING
[COURIER-1] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=10000ms
[COURIER-1] WAITING
[COURIER-2] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=51500ms
```

```
[CARGOGTU] SHUTDOWN_COMPLETE
==3181==
==3181== HEAP SUMMARY:
==3181==     in use at exit: 0 bytes in 0 blocks
==3181==   total heap usage: 15 allocs, 15 frees, 13,016 bytes allocated
==3181==
==3181== All heap blocks were freed -- no leaks are possible
==3181==
==3181== For lists of detected and suppressed errors, rerun with: -s
==3181== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

5.5 Test 5 – `-n 10 -i 20orders_mix.txt`, run multiple times

What it shows: across repeated invocations, `SHIFT_END completed=20` is identical every run; no `DELIVERY_START` id ever appears twice in a single run; the EXPRESS-before-STANDARD-before-ECONOMY ordering holds even under high concurrency where ten couriers are racing for the queue lock. The key invariant verified here is that the queue operation is genuinely atomic from the couriers' point of view.

Test 5 – console output

```
  Hw5 git:(main)  ./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=5
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Acar priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Burak_Turan priority=EXPRESS duration=4
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Koc priority=STANDARD duration=13
[CARGOGTU] ORDER_QUEUED id=9 recipient=Can_Ozkan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Arslan priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=11 recipient=Hasan_Kurt priority=STANDARD duration=7
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Yildiz priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=13 recipient=Emre_Dogan priority=EXPRESS duration=10
[CARGOGTU] ORDER_QUEUED id=14 recipient=Derya_Cetin priority=STANDARD duration=8
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kerem_Gunes priority=ECONOMY duration=14
[CARGOGTU] ORDER_QUEUED id=16 recipient=Melis_Aydin priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=17 recipient=Yusuf_Sari priority=STANDARD duration=16
[CARGOGTU] ORDER_QUEUED id=18 recipient=Ece_Kaplan priority=ECONOMY duration=22
[CARGOGTU] ORDER_QUEUED id=19 recipient=Omer_Polat priority=EXPRESS duration=9
[CARGOGTU] ORDER_QUEUED id=20 recipient=Deniz_Yaman priority=STANDARD duration=5
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-5] WAITING
[COURIER-5] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-6] WAITING
[COURIER-3] WAITING
[COURIER-7] WAITING
[COURIER-7] DELIVERY_START id=10 recipient=Merve_Arslan priority=EXPRESS
[COURIER-3] DELIVERY_START id=13 recipient=Emre_Dogan priority=EXPRESS
[COURIER-2] WAITING
[COURIER-8] WAITING
[COURIER-2] DELIVERY_START id=16 recipient=Melis_Aydin priority=EXPRESS
[COURIER-10] WAITING
[COURIER-10] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-6] DELIVERY_START id=7 recipient=Burak_Turan priority=EXPRESS
[COURIER-9] WAITING
[COURIER-9] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-4] WAITING
[COURIER-4] DELIVERY_START id=8 recipient=Elif_Koc priority=STANDARD
[COURIER-8] DELIVERY_START id=19 recipient=Omer_Polat priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=16 recipient=Melis_Aydin duration=1500ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=11 recipient=Hasan_Kurt priority=STANDARD
[COURIER-6] DELIVERY_COMPLETE id=7 recipient=Burak_Turan duration=2000ms
[COURIER-6] WAITING
[COURIER-6] DELIVERY_START id=14 recipient=Derya_Cetin priority=STANDARD
[COURIER-5] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=2500ms
[COURIER-5] WAITING
[COURIER-5] DELIVERY_START id=17 recipient=Yusuf_Sari priority=STANDARD
[COURIER-7] DELIVERY_COMPLETE id=10 recipient=Merve_Arslan duration=3000ms
[COURIER-7] WAITING
[COURIER-7] DELIVERY_START id=20 recipient=Deniz_Yaman priority=STANDARD
```

```
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-9] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-9] WAITING
[COURIER-9] DELIVERY_START id=6 recipient=Zeynep_Acar priority=ECONOMY
[COURIER-8] DELIVERY_COMPLETE id=19 recipient=Omer_Polat duration=4500ms
[COURIER-8] WAITING
[COURIER-8] DELIVERY_START id=9 recipient=Can_Ozkan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=13 recipient=Emre_Dogan duration=5000ms
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=12 recipient=Selin_Yildiz priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=11 recipient=Hasan_Kurt duration=3500ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=15 recipient=Kerem_Gunes priority=ECONOMY
[COURIER-7] DELIVERY_COMPLETE id=20 recipient=Deniz_Yaman duration=2500ms
[COURIER-7] WAITING
[COURIER-7] DELIVERY_START id=18 recipient=Ece_Kaplan priority=ECONOMY
[COURIER-10] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=6000ms
[COURIER-10] WAITING
[COURIER-6] DELIVERY_COMPLETE id=14 recipient=Derya_Cetin duration=4000ms
[COURIER-6] WAITING
[COURIER-4] DELIVERY_COMPLETE id=8 recipient=Elif_Koc duration=6500ms
[COURIER-4] WAITING
[COURIER-8] DELIVERY_COMPLETE id=9 recipient=Can_Ozkan duration=5500ms
[COURIER-8] WAITING
[COURIER-5] DELIVERY_COMPLETE id=17 recipient=Yusuf_Sari duration=8000ms
[COURIER-5] WAITING
[COURIER-9] DELIVERY_COMPLETE id=6 recipient=Zeynep_Acar duration=7500ms
[COURIER-9] WAITING
[COURIER-2] DELIVERY_COMPLETE id=15 recipient=Kerem_Gunes duration=7000ms
[COURIER-2] WAITING
[COURIER-1] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=10000ms
[COURIER-1] WAITING
[COURIER-3] DELIVERY_COMPLETE id=12 recipient=Selin_Yildiz duration=9000ms
[COURIER-3] WAITING
[COURIER-7] DELIVERY_COMPLETE id=18 recipient=Ece_Kaplan duration=11000ms
[COURIER-7] WAITING
[COURIER-3] SHIFT_OVER
[COURIER-5] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=107500ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

Test 5 – console output

```
  Hw5 git:(main)  ./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=5
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Acar priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Burak_Turan priority=EXPRESS duration=4
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Koc priority=STANDARD duration=13
[CARGOGTU] ORDER_QUEUED id=9 recipient=Can_Ozkan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Arslan priority=EXPRESS duration=6
```

```
[CARGOGTU] ORDER_QUEUED id=11 recipient=Hasan_Kurt priority=STANDARD duration=7
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Yildiz priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=13 recipient=Emre_Dogan priority=EXPRESS duration=10
[CARGOGTU] ORDER_QUEUED id=14 recipient=Derya_Cetin priority=STANDARD duration=8
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kerem_Gunes priority=ECONOMY duration=14
[CARGOGTU] ORDER_QUEUED id=16 recipient=Melis_Aydin priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=17 recipient=Yusuf_Sari priority=STANDARD duration=16
[CARGOGTU] ORDER_QUEUED id=18 recipient=Ece_Kaplan priority=ECONOMY duration=22
[CARGOGTU] ORDER_QUEUED id=19 recipient=Omer_Polat priority=EXPRESS duration=9
[CARGOGTU] ORDER_QUEUED id=20 recipient=Deniz_Yaman priority=STANDARD duration=5
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-4] WAITING
[COURIER-4] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=7 recipient=Burak_Turan priority=EXPRESS
[COURIER-8] WAITING
[COURIER-8] DELIVERY_START id=10 recipient=Merve_Arslan priority=EXPRESS
[COURIER-7] WAITING
[COURIER-7] DELIVERY_START id=13 recipient=Emre_Dogan priority=EXPRESS
[COURIER-10] WAITING
[COURIER-10] DELIVERY_START id=16 recipient=Melis_Aydin priority=EXPRESS
[COURIER-9] WAITING
[COURIER-9] DELIVERY_START id=19 recipient=Omer_Polat priority=EXPRESS
[COURIER-6] WAITING
[COURIER-6] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-5] WAITING
[COURIER-5] DELIVERY_START id=8 recipient=Elif_Koc priority=STANDARD
[COURIER-10] DELIVERY_COMPLETE id=16 recipient=Melis_Aydin duration=1500ms
[COURIER-10] WAITING
[COURIER-10] DELIVERY_START id=11 recipient=Hasan_Kurt priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Burak_Turan duration=2000ms
[COURIER-1] WAITING
[COURIER-1] DELIVERY_START id=14 recipient=Derya_Cetin priority=STANDARD
[COURIER-4] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=2500ms
[COURIER-4] WAITING
[COURIER-4] DELIVERY_START id=17 recipient=Yusuf_Sari priority=STANDARD
[COURIER-8] DELIVERY_COMPLETE id=10 recipient=Merve_Arslan duration=3000ms
[COURIER-8] WAITING
[COURIER-8] DELIVERY_START id=20 recipient=Deniz_Yaman priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-2] WAITING
[COURIER-2] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-9] DELIVERY_COMPLETE id=19 recipient=Omer_Polat duration=4500ms
[COURIER-9] WAITING
[COURIER-9] DELIVERY_START id=6 recipient=Zeynep_Acar priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-3] WAITING
[COURIER-3] DELIVERY_START id=9 recipient=Can_Ozkan priority=ECONOMY
[COURIER-7] DELIVERY_COMPLETE id=13 recipient=Emre_Dogan duration=5000ms
[COURIER-7] WAITING
[COURIER-7] DELIVERY_START id=12 recipient=Selin_Yildiz priority=ECONOMY
[COURIER-10] DELIVERY_COMPLETE id=11 recipient=Hasan_Kurt duration=3500ms
[COURIER-10] WAITING
[COURIER-10] DELIVERY_START id=15 recipient=Kerem_Gunes priority=ECONOMY
[COURIER-8] DELIVERY_COMPLETE id=20 recipient=Deniz_Yaman duration=2500ms
[COURIER-8] WAITING
[COURIER-8] DELIVERY_START id=18 recipient=Ece_Kaplan priority=ECONOMY
[COURIER-6] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=6000ms
[COURIER-6] WAITING
[COURIER-1] DELIVERY_COMPLETE id=14 recipient=Derya_Cetin duration=4000ms
[COURIER-1] WAITING
[COURIER-5] DELIVERY_COMPLETE id=8 recipient=Elif_Koc duration=6500ms
```

```
[COURIER-5] WAITING
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Can_Ozkan duration=5500ms
[COURIER-3] WAITING
[COURIER-4] DELIVERY_COMPLETE id=17 recipient=Yusuf_Sari duration=8000ms
[COURIER-4] WAITING
[COURIER-9] DELIVERY_COMPLETE id=6 recipient=Zeynep_Acar duration=7500ms
[COURIER-9] WAITING
[COURIER-10] DELIVERY_COMPLETE id=15 recipient=Kerem_Gunes duration=7000ms
[COURIER-10] WAITING
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=10000ms
[COURIER-2] WAITING
[COURIER-7] DELIVERY_COMPLETE id=12 recipient=Selin_Yildiz duration=9000ms
[COURIER-7] WAITING
[COURIER-8] DELIVERY_COMPLETE id=18 recipient=Ece_Kaplan duration=11000ms
[COURIER-8] WAITING
[COURIER-5] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-10] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=107500ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

6 Challenges & Solutions

Challenge 1 – Avoiding output line interleaving without losing throughput

Problem. When two couriers complete a delivery within microseconds of each other, naive `printf` calls can interleave at the byte level. The homework explicitly tests for this and penalises −20. The temptation is to reach for either fine-grained per-line locking or to serialise all logging through a single producer (e.g. a logger thread with a queue), but the first is what `printf` already partially provides on POSIX, which doesn't fully help, and the second adds an unnecessary thread and queue.

Solution. A dedicated `log_mutex` plus a single `log_printf` helper. The mutex is held for the entire formatted line and the trailing `fflush`. This ensures (a) the formatting and the write happen as one critical section, and (b) when stdout is redirected to a pipe, the line still emerges atomically because the flush is inside the lock. Holding a mutex during I/O is normally a bad pattern, but here the lines are tiny (≤ 100 bytes) and writes go to memory buffers, so contention is negligible. After this change, redirecting output through `tee | grep` produced no truncated or merged lines across many runs.

Challenge 2 – Honouring "a delivery cannot be interrupted" while still responding promptly to SIGINT

Problem. The homework requires two things that pull in opposite directions:

1. SIGINT must be observed and processed quickly (otherwise the user pressing Ctrl-C twice is a sign of unresponsiveness).
2. A courier currently sleeping out a delivery must finish that delivery – the simulated time must not be cut short.

A naive approach (block SIGINT in courier threads, leave it to main) misses the first requirement: `nanosleep` happily blocks for the full duration, the user waits. Setting `SA_RESTART` papers over the issue but again loses prompt response.

Solution. Leave `SA_RESTART` unset and wrap `nanosleep` in a small loop that resumes with the remaining time on `EINTR`. This single idea satisfies both requirements at once. Main's polling sleep also gets the same treatment by accident – when SIGINT fires, main's `nanosleep` returns immediately, the next iteration sees `g_shutdown_requested` set, and the shutdown sequence starts within microseconds. Couriers that are mid-delivery resume their sleep and complete the full simulated duration; the `DELIVERY_COMPLETE` line still reports the original duration, and the atomic counters are still updated correctly. The only behavioural difference from a no-SIGINT run is that no new deliveries are started after the flag is set.

A corollary is that `pq_pop_blocking` does not need any special SIGINT handling: by the time a courier reaches it after a finished delivery, main has already drained the queue and called `pq_shutdown`, so the pop returns `PQ_POP_SHUTDOWN` immediately and the thread exits cleanly. No interrupt logic is needed inside the queue itself.

Challenge 3 – Determining the precise moment to declare the shift over

Problem. Without SIGINT, when does the shift naturally end? Watching for an empty queue is wrong – the queue can be momentarily empty between two pops while a courier is still mid-delivery on the previous one. Watching for `pq_pop_blocking` to fail is also wrong because the natural end has no failure: couriers complete the last order and then go back to wait, indefinitely, on an empty queue.

Solution. Track completed deliveries against the known initial count. Main polls the `completed_orders` atomic counter and exits the wait loop when `completed_orders == n_orders`. At that moment, every order has been finished, so the queue is necessarily empty and no courier can possibly start another delivery. Calling `pq_shutdown` after the predicate becomes true wakes the now-idle couriers so they can log `SHIFT_OVER` and return.

This is clean because the `completed_orders` counter has a strict, simple meaning: "number of `DELIVERY_COMPLETE` lines logged so far". It is already being incremented for the statistics, so termination detection is essentially free – no new variable, no new lock.